第一讲教学设计：Web的核心脉络—HTTP协议与Tomcat的使命 (20分钟)

课程信息

- **课程名称**: Java Web应用开发 (Java Web Application Development)
- **教学方式**: 双语教学（英文授课，关键术语中英文对照）
- **教学主题**: Web的核心脉络：HTTP协议与Tomcat的使命
- **授课时长**: 20分钟
- **适用对象**: Java Web开发初学者，对Web开发有初步兴趣的学生

一、教学目标

知识目标

1. 理解HTTP协议（HyperText Transfer Protocol）是Web通信的基石，掌握其请求/响应（Request/Response）模型。
2. 了解HTTP请求和响应的主要组成部分（请求行Request Line/状态行Status Line、头Headers、体Body）。
3. 掌握Tomcat作为Servlet容器（Servlet Container）的核心作用，并了解其基本工作模式。

能力目标

1. 能够使用浏览器开发者工具观察和分析基本的HTTP请求与响应报文。
2. 能够独立完成Tomcat的下载、安装和启动。
3. 能够将一个简单的静态HTML页面成功部署到Tomcat服务器并访问。

素质目标

1. 建立对Web技术整体架构的宏观认识，理解各组件的协同作用。
2. 培养严谨的、遵循技术标准的工程思维。
3. **[思政融合]** 通过了解Web技术发展历程和我国网络建设成就，增强学生的国家意识和民族自豪感。

二、教学重点与难点

教学重点

- HTTP请求 (Request) 和响应 (Response) 的基本结构。
- Servlet容器 (Container) 的概念以及Tomcat在Web应用中的核心角色。
- Tomcat的安装、核心目录结构及启动/关闭操作。
- 在Tomcat中部署一个简单的Web应用。

教学难点

- 将抽象的HTTP协议概念与浏览器和服务器之间看得见的实际交互过程联系起来。
- 理解为什么需要Tomcat这样的容器来运行Java Web程序，而不能像普通Java程序一样直接运行。

三、教学内容与时间安排

第一部分：问题引入 - 一次网页请求的背后 (3分钟)

内容要点

1. **问题驱动 (Problem-Driven):**
   - 以一个学生最熟悉的场景开场："同学们，当我们在浏览器地址栏输入 `http://www.tyust.edu.cn` 并敲下回车时，屏幕上出现了太原科技大学的官网。这个看似简单的动作背后，究竟隐藏了怎样的技术之旅？我们的浏览器是如何"告诉"服务器它想要什么？服务器又是如何"回应"的？"
2. **高层概述 (High-Level Overview):**
   - 引出客户端（浏览器）和服务器（网站服务器）这两个核心角色。
   - 强调它们之间需要一种"通用语言"进行沟通，这种语言就是 **HTTP协议 (HyperText Transfer Protocol)**。
3. **思政融合 (Ideological Integration):**
   - 展示数据或图表，简要介绍我国作为全球最大互联网市场的地位和网络基础设施建设的巨大成就（如5G基站数量、光纤网络覆盖等）。
   - 引出学习Web技术不仅是掌握一门技能，更是未来参与"数字中国"、"网络强国"战略建设的基础，激发学生的学习热情和国家自豪感。

教学方法

- 提问互动，激发学生好奇心。
- 结合宏观数据和国家战略，提升学习的格局和意义。

第二部分：HTTP协议 - Web的通用语 (7分钟)

内容要点

1. **HTTP请求 (HTTP Request / The Request):** 讲解一个请求就像一封信，包含三个核心部分：
   - **请求行 (Request Line):** `GET /index.html HTTP/1.1` (告诉服务器Server：我想用GET方法Method，获取/index.html这个资源Resource，我遵循HTTP/1.1版本协议Protocol)
   - **请求头 (Request Headers):** `Host: www.tyust.edu.cn`，`User-Agent: Chrome/108...` (附加信息，如同信封上的寄件人信息)
   - **请求体 (Request Body):** (GET请求通常为空，为后面POST请求做铺垫，可比喻为空信件内容)
2. **HTTP响应 (HTTP Response / The Response):** 讲解服务器的回信也包含三部分：
   - **状态行 (Status Line):** `HTTP/1.1 200 OK` (告诉浏览器Browser：我用HTTP/1.1回复你，状态码Status Code是200，一切顺利)
   - **响应头 (Response Headers):** `Content-Type: text/html` (附加信息，告诉浏览器我给你的是一个HTML文件)
   - **响应体 (Response Body):** 实际的HTML代码 (信件的正文内容)。
3. **生动演示 (Live Demo):**
   - 打开任一网站（如学校官网），按下F12打开浏览器开发者工具，切换到 `Network` （网络）面板。
   - 刷新页面，实时展示捕获到的HTTP请求和响应报文，逐一对应讲解请求行、响应状态码（`200 OK`），以及 `Headers` 和 `Response` 内容，将抽象概念可视化。
   - 故意访问一个不存在的URL，演示 `404 Not Found` 状态码，加深学生理解。

教学方法

- 比喻教学法（信件模型）。
- 现场演示与理论讲解相结合，建立直观认识。

第三部分：Tomcat - Web应用的家 (7分钟)

内容要点

1. **为什么需要Tomcat?**
    - 提问："我们写的HTML文件，可以直接用浏览器打开。但如果我们的网页内容需要根据当前时间、用户信息等动态变化（即包含Java代码），浏览器能直接执行Java代码吗？"（引导学生回答：不能）
    - 引出结论：我们需要一个特殊的软件，它既能像Web服务器一样接收HTTP请求，又能运行Java代码。这个"超级服务器"就是 **Servlet容器**。
2. **什么是Tomcat?**
    - **定义:** Tomcat是Apache基金会下一个开源、免费的轻量级Web服务器和Servlet容器。
    - **核心使命:** 它的家就是用来装载和管理我们的Java Web应用的。它负责接收HTTP请求，找到并运行我们写的Java程序（Servlet），然后将程序生成的动态内容组合成一个完整的HTTP响应，再发送回浏览器。
3. **安装与核心目录 (Installation & Core Directories):**
    - 快速演示Tomcat的下载（官网）、解压（绿色免安装）。
    - 讲解核心目录，强调"各司其职"：
        - `bin`：**启动器** - 存放启动和关闭脚本（`startup.bat` / `.sh`，`shutdown.bat` / `.sh`）。
        - `conf`：**配置中心** - 存放服务器配置文件（`server.xml`）。
        - `webapps`：**公寓楼** - 存放我们开发的Web应用的地方，每个应用一个文件夹。
        - `logs`：**日志室** - 记录服务器运行日志，排错必备。
4. **部署与运行 (Deploy & Run):**
    - **第一步：创建应用** - 在桌面创建一个 `myapp` 文件夹。
    - **第二步：编写主页** - 在 `myapp` 文件夹内创建一个 `index.html` 文件，内容为 `<h1>Hello, Tomcat!</h1>`。
    - **第三步：入住公寓** - 将整个 `myapp` 文件夹复制到Tomcat的 `webapps` 目录下。
    - **第四步：启动服务** - 进入 `bin` 目录，双击 `startup.bat` 启动Tomcat。
    - **第五步：验证访问** - 在浏览器访问 `http://localhost:8080/myapp/index.html`，看到 "Hello, Tomcat!" 字样，部署成功！

教学方法

- 设问-解答式教学。
- 分步演示，指令清晰，确保学生能跟上。

第四部分：课堂总结与展望 (3分钟)

总结要点

1. **通信靠协议**: 浏览器和服务器通过HTTP协议进行"对话"。
2. **对话有格式**: HTTP通信包含请求和响应两部分，各有固定格式。
3. **动态靠容器**: Java Web应用不能独立运行，必须住在像Tomcat这样的Servlet容器里。

4. **部署很简单**: 我们只需将Web应用文件夹放入Tomcat的 `webapps` 目录即可完成部署。

展望 (承上启下)

- "今天，我们成功地让Tomcat为我们展示了一个静态页面。但这仅仅是Web世界的冰山一角。真正的Web应用是动态的、可交互的。"
- "下一节课，我们将亲手编写第一个Servlet程序，让我们的Web应用不再只是一个'展示板'，而是能真正'听懂'用户请求并做出动态回应的智能应用。敬请期待！"

四、教学方法与手段

教学方法

1. **问题驱动法 (Problem-Driven Method)**: 以实际问题引导学习
2. **讲授法 (Lecturing)**: 系统讲解核心概念和原理
3. **演示法 (Demonstration)**: 现场演示HTTP请求和Tomcat部署
4. **比喻法 (Analogy)**: 用生活化比喻降低理解难度
5. **双语教学法 (Bilingual Teaching)**:
    - 课堂采用英文授课，关键技术术语提供中英文对照
    - 重要概念首次出现时给出英文表达和中文释义
    - 鼓励学生使用英文术语回答问题
    - 代码注释和变量命名使用规范英文

教学手段

- 多媒体课件 (Marp Slides)
- 浏览器开发者工具 (Browser DevTools)
- Tomcat服务器实例 (Tomcat Server Instance)

五、教学评价

- **过程性评价**: 课堂提问的回答情况；能否在指导下正确解读浏览器开发者工具中的信息。
- **终结性评价**: 能否用自己的话简述一次HTTP请求到响应的完整过程；能否独立将一个HTML文件部署到Tomcat并成功访问。

六、课后拓展

思考题

1. HTTP的GET请求和POST请求在使用场景上有什么主要区别？
2. 除了Tomcat，你还知道哪些其他的Web服务器或Servlet容器？
3. `404 Not Found` 状态码是什么意思？你还见过或听说过哪些常见的HTTP状态码？

实践作业

1. 在自己的电脑上成功安装并启动Tomcat。
2. 创建一个简单的个人介绍HTML页面（可以包含图片和超链接），将其命名为 `aboutme` 项目，并成功部署到Tomcat，通过 `http://localhost:8080/aboutme/` 访问。